

Final

Graded

Student
Myron Weng
Total Points
69 / 100 pts

Question 1

Problem 1

20 / 20 pts

+ 20 pts Point adjustment

Question 2

Problem 2

16 / 20 pts

+ 0 pts Correct

+ 16 pts Each subpart is worth 5 marks

(a) 5

(b) 1

(c) 5

(d) 5

Question 3

Problem 3

8 / 20 pts

+ 0 pts Correct

- 12 pts a)

b) Correct

c) Wrong, but right approach, could've found D

Question 4

Problem 4

18 / 20 pts

- 0 pts Correct

- 2 pts Incorrect concatenation

- 1 pt incorrect indexing of X and y

- 1 pt incorrect gen_data definition

- 2 pts text transformation not used

- 1 pt Layer 1 declaration incorrect

- 1 pt Layer 2 declaration incorrect

- 2 pts Incorrect final data

- 1 pt Coding error 1

- 2 pts Incorrect generation of target

- 1 pt Layer 1 not used in forward

- 1 pt Loop range incorrect

- 10 pts Part A missing

- 2 pts Incorrect forward pass.

- 5 pts gen_data incorrect logic

- 10 pts Part B missing

- 5 pts pricepred incorrect logic

- 1 pt wrong activation used

(a) 8, (b) 10

Question 5

Problem 5

7 / 20 pts

+ 0 pts Correct

- 13 pts a) Wrong

b) Almost correct. Minor mistkaes

ECE-GY 6143: Introduction to Machine Learning
Final, Fall 2025

Prof. Sundeep Rangan

Name: Myron Weng

NetID: mw4908

Instructions:

- Exam is closed-book, no calculators or electronic aids.
- You are permitted four double-sided cheat sheets.
- Exam time is 150 minutes
- Write answers on the pages. If you run out of pages, please ask the instructor or course assistant for more pages.

Good luck and Happy Holidays!

1. *Neural networks*: Consider a neural network with a scalar input x and scalar output $\hat{y}$:

$$h_j = W_j^H x + b_j^H, \quad u_j = \max\{0, h_j\}, \quad j = 1, \dots, N_H \quad (1a)$$

$$\hat{y} = \sum_{j=1}^{N_H} W_j^O u_j + b^O. \quad (1b)$$

- Draw u_j vs. x for the case when $W_j^H > 0$. Indicate the slopes and intercepts of any lines to ensure that the line is fully specified.
- Repeat part (a) for the case when $W_j^H < 0$.
- Find parameters of the network such that the mapping $\hat{y}$ vs. x is as shown in Fig. 1 using $N_H = 2$ hidden units. Hint: You can select $W_j^O = 1$ or -1 for all j , but be careful how you select the sign.

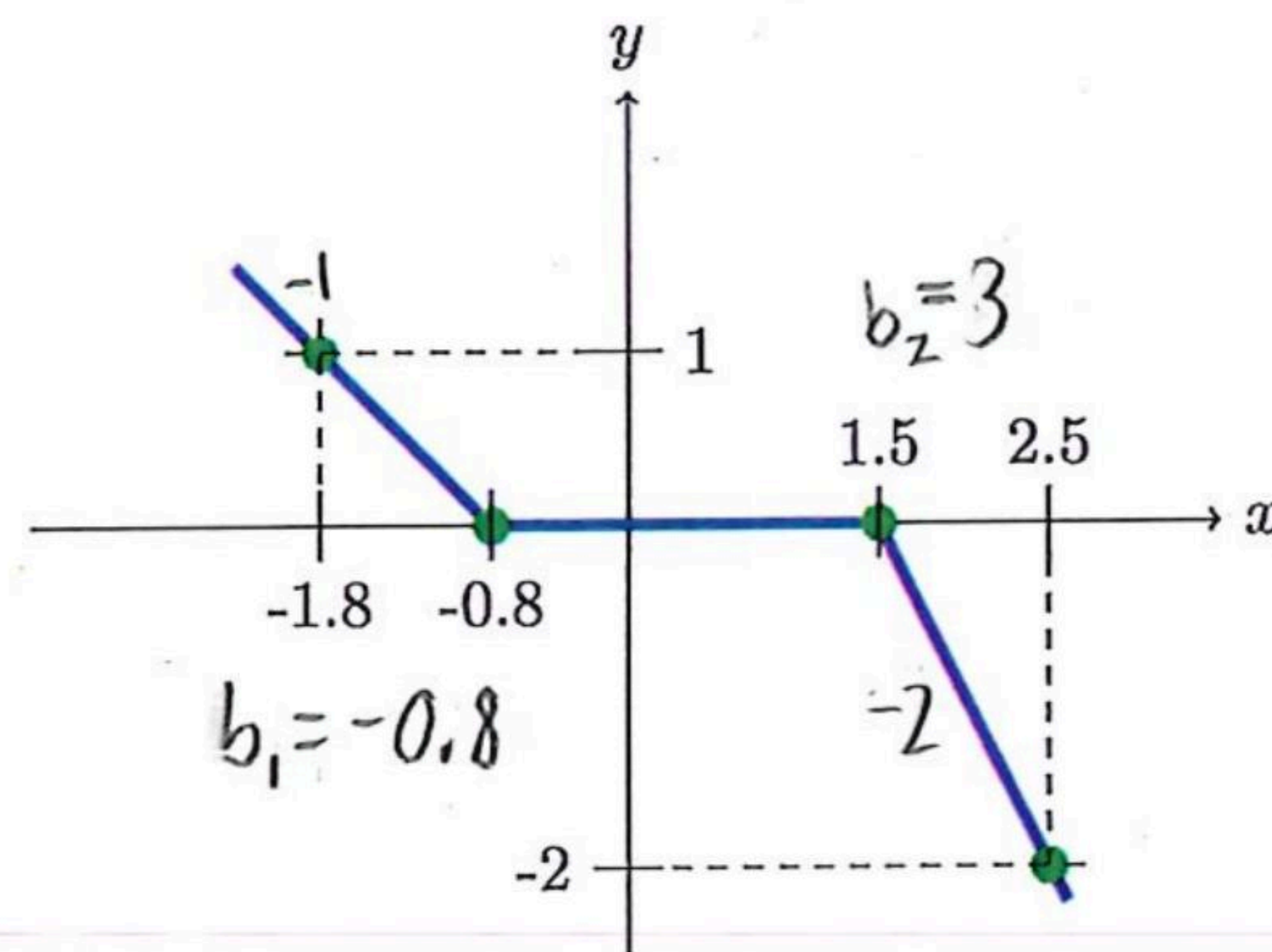
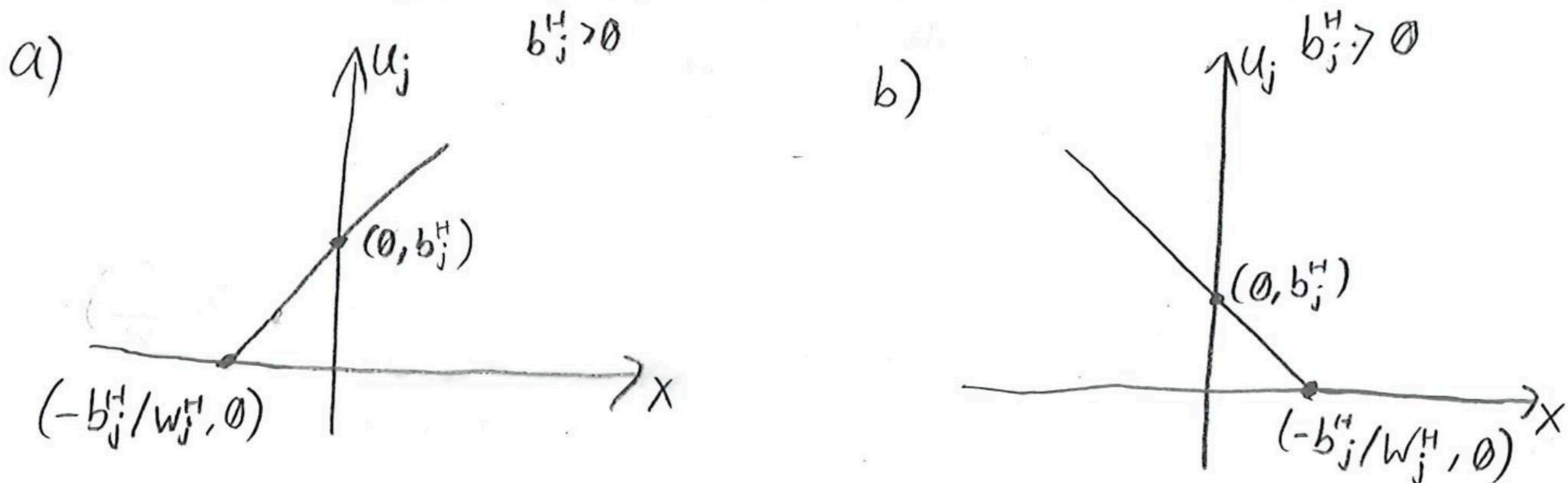


Figure 1: Graph of $\hat{y}$ vs. x . The curve goes through the four green points.



c) $h_1 = -x - 0.8; h_2 = -2x + 3; W_1^O = 1; W_2^O = -1$

Use for Problem 1

2. CNNs: Suppose that each sample for the input of an imaging system consists of gray scale images at $n_{\text{times}}=2$ different times.

- If each image is a gray scale with shape $(n_{\text{row}}, n_{\text{col}})=(384, 512)$, what is the shape of the input tensor of $n_{\text{samp}}=100$ samples.
- Suppose the first layer of an image processing network is a 2D convolutional layer with kernels of size $(\text{ker_size}, \text{ker_size})=(3, 3)$ and $n_{\text{out}}=10$ output channels, what is the shape of the kernels weights, w , and the biases, b ?
- Suppose the input tensor from part (a) is given to the network. What is the shape of the tensor at the output of the first layer? Assume the convolutional layer runs in same mode so the dimensions of each channel are not changed.
- Give one example of a kernel weight w that measures the difference between pixels (i, j) on the image in the first time with pixels $(i+1, j+1)$ on the second time. That difference should be on output channel 0.

a) $(100, 384, 512, 2)$
 $\uparrow \quad \uparrow \quad \uparrow \quad \uparrow$
 $n_{\text{samp}} \quad n_{\text{row}} \quad n_{\text{col}} \quad n_{\text{times}}$

b) $W: (2, 3, 3, 10) \ \& \ b: (2, 1, 10)$
 $\uparrow \quad \uparrow \quad \uparrow \quad \uparrow$
 $n_{\text{times}} \quad \text{ker_size} \quad \text{ker_size} \quad n_{\text{out}}$

c) $(100, 384, 512, 10)$
 $\uparrow$
 n_{out}

d) $W = \left[\begin{bmatrix} 1 & 0 & 0 \\ 0 & 0 & 0 \\ 0 & 0 & 0 \end{bmatrix}, \begin{bmatrix} 0 & 0 & 0 \\ 0 & -1 & 0 \\ 0 & 0 & 0 \end{bmatrix} \right]$

Use for Problem 2.

3. *PCA*: You want to train a model using PCA as a pre-processing step. You are given the following functions:

- `pca = PCA(ncomp)`: Creates a PCA class using the top `ncomp` components
- `pca.fit(X)`: Fits the PCA to find the mean and principal components of the data
- `pca.mean`: Mean vector found from the `pca.fit` function.
- `Z = pca.transform(X)`: Transforms data `X` to `ncomp` components.
- `mod.fit(Z,y)`: Fits a model from training data
- `yhat = mod.predict(Z)`: Predicts a model from features `Z`

(a) Suppose a data sample x_i has a K -term PC approximation:

$$\hat{x}_i = \mu + \sum_{j=1}^K z_{ij} v_j,$$

where μ is the mean of the data, v_j are the PC components and z_{ij} are the coefficients. Write an expression for the approximation error

$$d_i = \|x_i - \hat{x}_i\|^2$$

in terms of x_i , μ and z_{ij} , $j = 1, \dots, K$. Your expression should not use the PC vectors v_j .

- (b) Given training data, `Xtr,ytr`, and test data, `Xts,yts`, write python code to fit the model `mod` with PCA pre-processing and evaluate the MSE on the test data. Assume `ncomp` is specified.
- (c) Write python code that, given data `X,y`, finds data `X1,y1`, with all the samples with $d_i > t$ removed. Here t is some given threshold. You may use the numpy command:

```
I = np.where(cond)[0] % Find indices i where cond[i]==True
```

b)

```
pca=PCA(ncomp)
pca.fit(Xtr)
Ztr=pca.transform(Xtr)
mod.fit(Ztr,ytr)
Zts=pca.transform(Xts)
yhat=mod.predict(Zts)
MSE=np.mean((yhat-yts)**2)
```

c) # each row of D contains d_i

```
I=np.where(D>t)
X1=X[I] # select rows w/ indices
y1=y[I]
```


Use for Problem 3.

$$a) d_i = \left\| \vec{x}_i - \vec{\mu} - \sum_{j=1}^K \sum_{l=1}^N z_{ij} (\vec{x}_i - \vec{\mu}) \right\|^2$$

4. *Text embeddings.* We wish to train a neural network to predict stock market price based on past prices and tweets about the stock. For training data we are given:

- **price:** A vector of n stock market values
- **tweet:** A list of n strings representing tweets about the stock
- **emb = text_transform(text):** A function that takes a list of strings and creates an array of embeddings. Each embedding is of dimension `emb_dim`.

We want to predict `price[i+1]` from `price[i-2]`, `price[i-1]`, `price[i]` and the embedding of `tweet[i]` where i ranges from 2 to $n-2$.

(a) Write a function:

```
def gen_data(price, tweet):  
    ...  
    return X, y
```

to convert the prices and tweets to an array, `X`, of inputs and vector, `y`, of predicted outputs of the model. Each row of `X` should have three prices and the embedding of the tweet. You may use:

```
np.concatenate((a,b)) # Concatenates two vectors a and b
```

Although there are solutions that do not use this concatenate function.

(b) Complete the following PyTorch code that makes the prediction using two fully-connected layers: a hidden layer, `fc1`, and an output layer, `fc2`. The hidden layer should have `nhid` hidden units and ReLU activations. Make sure the dimensions of the inputs and outputs of the layers are correct.

```
class PricePred(nn.Module):  
    def __init__(self, nhid=64):  
        super().__init__()  
        self.fc1 = ...  
        self.fc2 = ...  
  
    def forward(self, x):  
        ...  
        return yhat
```

You may use:

- `nn.Linear(nin, nout)`: Fully-connected layer with `nin` inputs and `nout` outputs.
- `F.ReLU(x)`: Takes ReLU activations of `x`.

Use for Problem 4.

```
a) def gen_data(price, tweet):  
    n = price.shape[0]  
    p = np.array([price[0], price[1], price[2]])  
    X = concatenate((p, text_transform(tweet[2])))  
    for i in range(3, n-1):  
        p_i = np.array([price[i-2], price[i-1], price[i]])  
        X_i = concatenate((p_i, text_transform(tweet[i])))  
        X = np.vstack(X, X_i)  
    y = mod.predict(X)  
    return X, y
```

```
b) def __init__(self, nhid=64):  
    super().__init__()  
    self.fc1 = nn.Linear(emb_dim+4, nhid)  
    self.activation = F.ReLU(X)  
    self.fc2 = nn.Linear(nhid, 1)  
    self.activation = F.ReLU(X)  
    def forward(self, X):  
        Zh1 = self.fc1(X)  
        uh1 = self.activation(Zh1)  
        Zo = self.fc2(uh1)  
        Uo = self.activation(Zo)  
        return Uo
```


5. *Clustering.* A system performs clustering with a cosine similarity metric between two vectors x and $y \in \mathbb{R}^d$:

$$S(x, y) = \frac{x^T y}{\|x\| \|y\|} = \cos \theta, \text{ where } \theta \text{ is angle b/w } \vec{x} \text{ \& } \vec{y}$$

- (a) Five cluster centers, μ_0 to μ_4 , are shown in Fig. 2. Describe and draw the region corresponding to the points $x = (x_0, x_1)$ most similar to cluster center μ_2 .
- (b) You are given a numpy arrays x and cluster μ of samples and cluster centers. Write python code to compute a similarity matrix where $S[i, j]$ is the similarity of sample i with cluster center j .

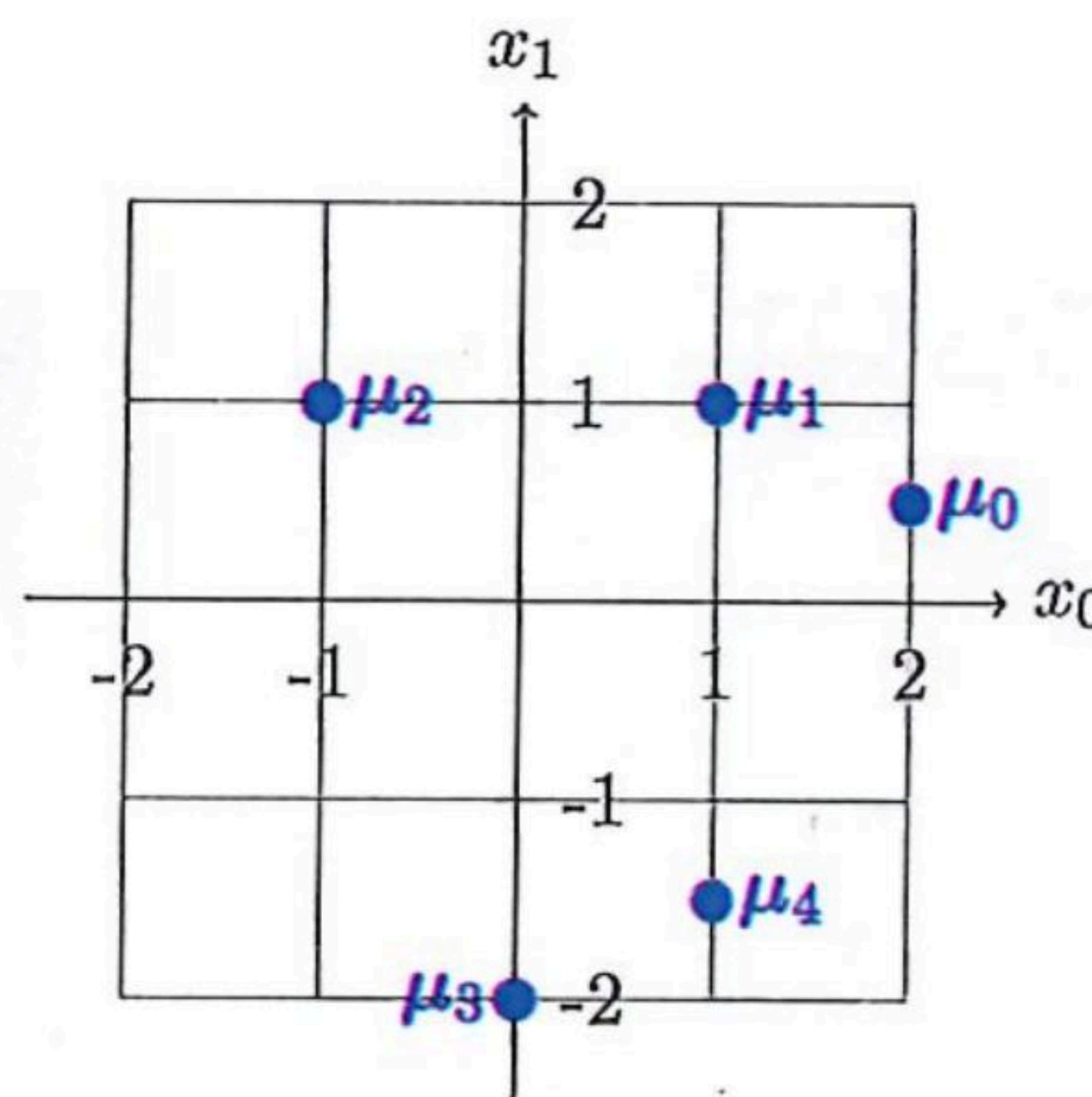
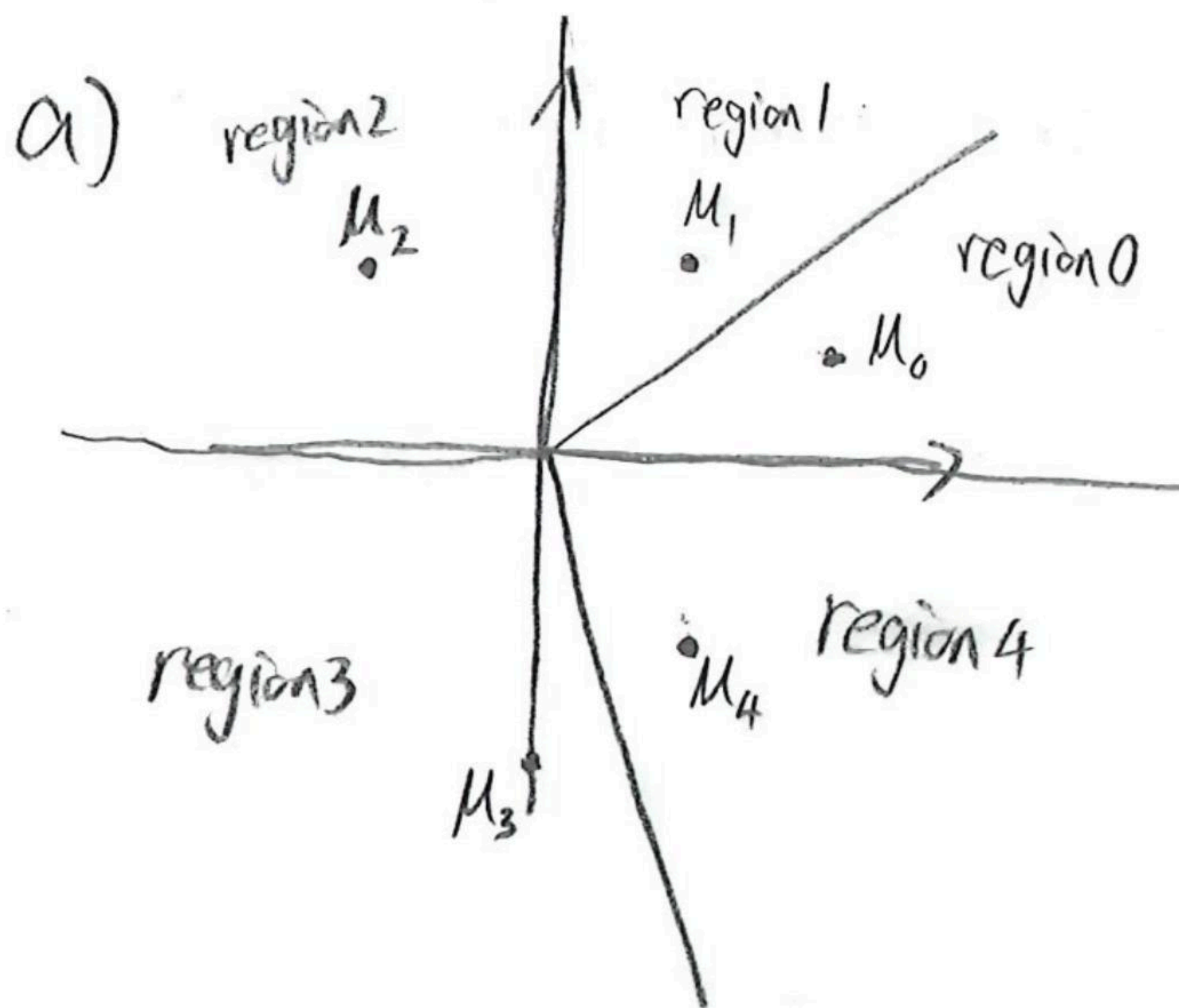


Figure 2: Five cluster centers, μ_0 to μ_4 , are shown as blue circles.



each region is a sector;
 $\vec{x}$ belongs to a region if
 its angle between $\vec{\mu}_i$ is
 smaller than that between the
 $\vec{\mu}_s$ of other regions

Use for Problem 5.

~~X~~ & mu should
have same number
of columns

```
b) m = X.shape[0] # how many vectors  
    n = mu.shape[0] # how many clusters  
    S = np.zeros((m, n))  
    for i in range(m):
```

```
        for j in range(n):  
            S[i, j] = (X[i] * mu[j]) / (np.sum(X[i]**2) * np.sum(mu[j]**2))
```